4 性能分析中的术语和指标

像许多工程学科一样，性能分析在使用特殊术语和指标方面非常重要。对于初学者来说，查看由分析工具如 Linux `perf`或 Intel VTune Profiler 生成的性能分析文件可能会感到非常困难。这些工具涉及许多复杂的术语和指标，但如果您打算进行任何严肃的性能工程工作，这些内容是“必须了解”的。

既然我们提到了 Linux `perf`，让我们简要介绍一下这个工具，因为我们在本章和后续章节中有许多使用它的示例。Linux `perf`是一个性能分析器，您可以使用它来查找程序中的热点，收集各种低级 CPU 性能事件，分析调用堆栈等等。我们将在整本书中广泛使用 Linux `perf`，因为它是最流行的性能分析工具之一。我们偏爱展示 Linux `perf`的另一个原因是因为它是开源软件，这使得热衷的读者可以探索现代分析工具内部发生的机制。这对于学习本书中提出的概念特别有用，因为基于 GUI 的工具（如 Intel® VTune™ Profiler）往往会隐藏所有复杂性。我们将在第 7 章中对 Linux `perf`进行更详细的概述。

本章是对性能分析中使用的基本术语和指标的简要介绍。我们将首先定义诸如已退役/执行指令、IPC/CPI、μops、核心/参考时钟、缓存失效和分支误预测等基本概念。然后，我们将看到如何测量系统的内存延迟和带宽，并介绍一些更高级的指标。最后，我们将对四种行业工作负载进行基准测试，并查看收集到的指标。

4.1 已退役 (Retired) vs. 已执行 (Executed) 指令

现代处理器通常执行的指令数量比程序流程所需的要多。这是因为一些指令是以推测性的方式执行的，如 Section 3.3.3中所讨论的。对于大多数指令，CPU 在结果可用时提交，所有先前的指令都已退役 (retired)。但对于以推测性方式执行的指令，CPU 会保留它们的结果，而不立即提交它们的结果。当推测结果被证明是正确时，CPU 会解除此类指令的阻塞并正常进行。但当推测结果被证明是错误时，CPU 会丢弃推测指令所做的所有更改，并不会退役它们。因此，CPU 处理的指令可以被执行但不一定被退役。考虑到这一点，我们通常可以预期执行的指令数量高于已退役的指令数量。

但有一个例外。某些指令被识别为惯用语，并且在没有实际执行的情况下被解析。其中一些示例是 NOP、移动消除和清零，如 Section 3.8.2中所讨论的。这些指令不需要执行单元，但仍然被退役。因此，从理论上讲，可能存在已退役指令数量高于已执行指令数量的情况。

大多数现代处理器都有一个性能监视计数器（PMC），用于收集已退役指令的数量。虽然没有性能事件来收集已执行的指令，但有一种方法可以收集已执行和已退役的微操作，我们很快将会看到。可以通过运行以下命令轻松获取已退役指令的数量，使用 Linux `perf`：

```
$ perf stat -e instructions ./a.exe
  2173414  instructions  #    0.80  insn per cycle
# 或者简单地执行：
$ perf stat ./a.exe
```

4.2 CPU 利用率

CPU 利用率是在一段时间内 CPU 处于忙碌状态的百分比。从技术上讲，当 CPU 不运行内核的`idle`线程时，CPU 被认为是被利用的。

$$CPU\ Utilization = \frac{CPU_CLK_UNHALTED.REF_TSC}{TSC},$$

其中，CPU_CLK_UNHALTED.REF_TSC 计算了核心处于非停顿状态时的参考周期数，TSC 代表时间戳计数器（在 Section 2.6中讨论过），它始终在运行。

如果 CPU 利用率低，通常意味着应用程序性能较差，因为 CPU 浪费了一部分时间。然而，高 CPU 利用率并不总是好性能的指标。这仅仅是系统正在进行一些工作的迹象，但并不表示正在做什么：即使 CPU 由于等待内存访问而被阻塞，它仍然可能被高度利用。在多线程环境中，线程在等待资源继续进行时也可以自旋。稍后在 Section 13.2中，我们将讨论并行效率指标，特别是“有效 CPU 利用率”，该指标过滤了自旋时间。

Linux perf会自动计算系统上所有 CPU 的 CPU 利用率：

```
$ perf stat -- a.exe
  0.634874  task-clock (msec) #    0.773 CPUs utilized
```

4.3 CPI 和 IPC

这两个是两个基本指标，分别代表：

- 每条指令周期数 (CPI) - 平均执行一条指令所需的周期数。

$$IPC = \frac{INST_RETIRED.ANY}{CPU_CLK_UNHALTED.THREAD},$$

其中，INST_RETIRED.ANY 计算已完成指令的数量，CPU_CLK_UNHALTED.THREAD 计算线程不在中止状态时的核心周期数。

- 每周期指令数 (IPC) - 平均每个周期完成的指令数。

$$CPI = \frac{1}{IPC}$$

使用哪一个取决于个人喜好。本书的主要作者更喜欢使用 IPC，因为它更容易比较。使用 IPC，我们希望每个周期尽可能多地执行指令，因此 IPC 越高越好。使用 CPI 则相反：我们希望每个指令的周期越少越好，所以 CPI 越低越好。使用“越高越好”的指标进行比较更简单，因为您不必每次都进行心理反转。在本书的其余部分，我们将主要使用 IPC，但再次申明，使用 CPI 也没有错。

IPC 和 CPU 时钟频率之间的关系非常有趣。从广义上讲，性能= 工作 / 时间，我们可以将工作表示为指令数，时间表示为秒。程序运行的秒数是 总周期/ 频率：

$$\text{性能} = \frac{\text{指令} \times \text{频率}}{\text{周期}} = IPC \times \text{频率}$$

正如我们看到的，性能与 IPC 和频率成正比。如果我们增加这两个指标中的任何一个，程序的性能就会提高。

从基准测试的角度来看，IPC 和频率是两个独立的指标。我们见过许多工程师错误地将它们混为一谈，认为如果增加频率，IPC 也会上升。但事实并非如此，IPC 将保持不变。如果您将处理器的时钟设置为 1 GHz 而不是 5Ghz，您仍然会拥有相同的 IPC。这非常令人困惑，尤其是因为 IPC 与 CPU 时钟密切相关。频率只告诉单个时钟的快慢，而 IPC 不考虑时钟变化的速度，它计算每个周期完成的工作量。因此，从基准测试的角度来看，IPC 完全取决于处理器的设计，与频率无关。乱序内核通常具有比顺序内核更高的 IPC。当您增加 CPU 缓存的大小或改进分支预测时，IPC 通常会上升。

现在，如果您问硬件架构师，他们肯定会告诉您 IPC 和频率之间存在依赖关系。从 CPU 设计的角度来看，您可以故意降低处理器时钟频率，这将使每个周期更长，并且可以在每个周期中塞入更多工作。最终，您将获得更高的 IPC

但更低的频率。硬件供应商以不同的方式处理性能公式。例如，英特尔和 AMD 芯片通常具有非常高的频率，最近的英特尔 13900KS 处理器开箱即用即可提供 6Ghz 的睿频频率，无需超频。另一方面，Apple M1/M2 芯片的频率较低，但通过更高的 IPC 进行补偿。较低的频率有助于降低功耗。另一方面，更高的 IPC 通常需要更复杂的设计、更多的晶体管和更大的芯片尺寸。我们这里不会讨论所有设计权衡，因为这涉及另一个主题。我们将在 Chapter 14 中讨论 IPC 和频率的未来发展。

IPC 对于评估硬件和软件效率都非常有用。硬件工程师使用此指标比较不同供应商的不同 CPU 代和 CPU。由于 IPC 是衡量微架构性能优劣的指标，因此工程师和媒体会使用它来表达最新 CPU 比上一代性能的提升。但要进行公平的比较，您需要在相同频率下运行这两个系统。

IPC 也是评估软件的有用指标。它可以让您直观地了解应用程序中的指令如何快速地穿过 CPU 流水线。稍后，您将在本章中看到几个具有不同 IPC 的生产应用程序。内存密集型应用程序通常以低 IPC (0-1) 为特征，而计算密集型工作负载往往具有高 IPC (4-6)。

Linux `perf` 用户可以通过运行以下命令测量其工作负载的 IPC：

```
$ perf stat -e cycles,instructions -- a.exe
2369632 cycles
1725916 instructions # 0,73 insn per cycle
# 或更简单地：
$ perf stat ./a.exe
```

4.4 微操作

具有 x86 架构的微处理器将复杂的 CISC 类指令转换为简单的 RISC 类微操作，缩写为 μops。例如，像 `ADD rax, rbx` 这样的简单加法指令只会生成一个 μop，而更复杂的指令比如 `ADD rax, [mem]` 可能生成两个：一个用于从 `mem` 内存位置读取到临时（未命名）寄存器，另一个用于将其添加到 `rax` 寄存器。指令 `ADD [mem], rax` 会生成三个 μops：一个用于从内存读取，一个用于相加，一个用于将结果写回内存。

将指令分割成微操作的主要优点是 μops 可以执行：

- **乱序**: 考虑 `PUSH rbx` 指令，它将栈指针减少 8 字节，然后将源操作数存储在栈顶。假设在解码后 `PUSH rbx` 被“破解”成两个依赖的微操作：

```
SUB rsp, 8
STORE [rsp], rbx
```

通常，函数序言通过使用多个 `PUSH` 指令保存多个寄存器。在我们的例子中，下一个 `PUSH` 指令可以在前一个 `PUSH` 指令的 `SUB` μop 完成后开始执行，而不必等待现在可以异步执行的 `STORE` μop。

- **并行**: 考虑 `HADDPD xmm1, xmm2` 指令，它将在 `xmm1` 和 `xmm2` 中对两个双精度浮点数进行求和（减少），并将两个结果存储在 `xmm1` 中，如下所示：

```
xmm1[63:0] = xmm2[127:64] + xmm2[63:0]
xmm1[127:64] = xmm1[127:64] + xmm1[63:0]
```

微代码化此指令的一种方法是执行以下操作：1) 减少 `xmm2`并将结果存储在 `xmm_tmp1[63:0]` 中，2) 减少 `xmm1`并将结果存储在 `xmm_tmp2[63:0]` 中，3) 将 `xmm_tmp1` 和 `xmm_tmp2` 合并到 `xmm1` 中。总共三个 μops。请注意，步骤 1) 和 2) 是独立的，因此可以并行完成。

尽管我们刚刚讨论了如何将指令分割成更小的部分，但有时 μops 也可以融合在一起。现代 CPU 中有两种类型的融合：

- 微融合: 融合来自同一机器指令的 μops。微融合只能应用于两种类型的组合：内存写操作和读改操作。例如：

```
add eax, [mem]
```

这条指令中有两个 μops：1) 读取内存位置 mem，2) 将其添加到 eax。使用微融合，在解码步骤中将两个 μops 融合成一个。

- 宏融合: 融合来自不同机器指令的 μops。在某些情况下，解码器可以将算术或逻辑指令与 subsequent 条件跳转指令融合成单个计算和分支 μop。例如：

```
.loop:
  dec rdi
  jnz .loop
```

使用宏融合，将来自 DEC 和 JNZ 指令的两个 μops 融合成一个。

微融合和宏融合都可以节省从解码到退役 (retired) 的所有流水线阶段的带宽。融合操作在重新排序缓冲区 (ROB) 中共享单个条目。当一个融合的 μop 只使用一个条目时，ROB 的容量得到更好的利用。这样的一个融合的 ROB 条目稍后会分派到两个不同的执行端口，但作为单个单元再次退役 (retired)。读者可以 [Fog, 2012] 中了解更多关于 μop 融合的信息。

要收集应用程序发出的、执行的和退役 (retired) 的 μops 数量，您可以使用 Linux perf，如下所示：

```
$ perf stat -e uops_issued.any,uops_executed.thread,uops_retired.slots -- ./a.exe
2856278 uops_issued.any
2720241 uops_executed.thread
2557884 uops_retired.slots
```

指令被分解成微操作的方式可能会随着 CPU 世代的不同而有所差异。通常，用于一条指令的 μops 数量越少，意味着硬件对其支持越好，并且可能具有更低的延迟和更高的吞吐量。对于最新的 Intel 和 AMD CPU，绝大多数指令都会生成恰好一个 μop。有关最近微架构中 x86 指令的延迟、吞吐量、端口使用情况和 μops 数量，可以参考 uops.info: https://uops.info/table.html[48] 网站。

4.5 流水线槽

另一个一些性能工具使用的重要指标是**流水线槽 (pipeline slot)** 的概念。流水线槽代表处理一个微操作所需的硬件资源。图 25 展示了一个每周期有 4 个分配槽的 CPU 的执行流水线。这意味着核心可以在每个周期将执行资源（重命名的源和目标寄存器、执行端口、ROB 条目等）分配给 4 个新的微操作。这样的处理器通常被称为 **4 宽机器 (4-wide machine)**。在图中连续的六个周期中，只利用了一半可用槽位。从微架构的角度来看，执行此类代码的效率只有 50%。

英特尔的 Skylake 和 AMD Zen3 内核具有 4 宽分配。英特尔的 SunnyCove 微架构采用 5 宽设计。截至 2023 年，最新的 Goldencove 和 Zen4 架构都采用 6 宽分配。Apple M1 的设计没有官方披露，但测得为 8 宽。[49]

流水线槽是自顶向下微架构分析 (见 Section 6.1) 的核心指标之一。例如，前端受限和后端受限指标由于各种瓶颈而表示为未使用的流水线槽的百分比。

[48] 指令延迟和吞吐量 - https://uops.info/table.html
[49] Apple 微架构研究 - https://dougallj.github.io/applecpu/firestorm.html

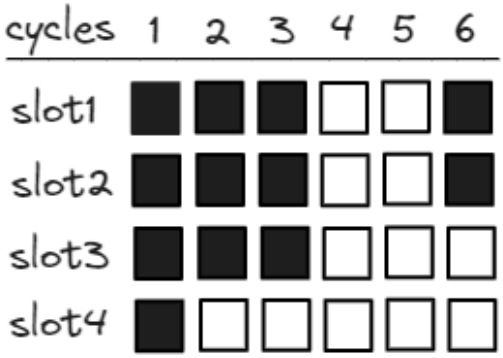


Figure 25: 4 宽 CPU 的流水线图

4.6 核心周期与参考周期

大多数 CPU 都使用时钟信号来控制它们的顺序操作。时钟信号由外部发生器产生，每秒提供一致数量的脉冲。时钟脉冲的频率决定了 CPU 执行指令的速率。因此，时钟越快，CPU 每秒执行的指令就越多。

$$\text{频率} = \frac{\text{时钟周期数}}{\text{时间}}$$

大多数现代 CPU，包括英特尔和 AMD 的 CPU，没有固定的运行频率。相反，它们实现了动态频率缩放，在英特尔的 CPU 中称为 Turbo Boost，在 AMD 处理器中称为 Turbo Core。它使 CPU 能够动态增加和减少频率。降低频率可以减少功耗，但会牺牲性能，增加频率可以提高性能，但会牺牲节能。

核心时钟周期计数器计算的是 CPU 核心实际运行的频率下的时钟周期数，而不是外部时钟（参考周期）。让我们看一下在 Skylake i7-6000 处理器上运行单线程应用程序的实验，它的基础频率为 3.4 GHz：

```
$ perf stat -e cycles,ref-cycles ./a.exe
  43340884632  cycles    # 3.97 GHz
  37028245322  ref-cycles # 3.39 GHz
      10.899462364 seconds time elapsed
```

指标`ref-cycles`统计的周期数是如果没有频率缩放的情况下。设置的外部时钟频率为 100 MHz，如果我们乘以时钟倍频，我们将得到处理器的基础频率。Skylake i7-6000 处理器的时钟倍频为 34：这意味着对于每个外部脉冲，当 CPU 运行在基础频率上时，它执行 34 个内部周期。

指标`cycles`统计的是真实的 CPU 周期数，即考虑了频率缩放。使用上述公式，我们可以确认平均运行频率为`43340884632个周期 / 10.899秒 = 3.97 GHz`。当您比较两个版本的小段代码的性能时，以时钟周期计时比以纳秒计时更好，因为您避免了时钟频率上下波动的问题。

4.7 缓存失效

正如在 Section 3.6中讨论的那样，任何在特定级别的缓存中缺失的内存请求都必须由更高级别的缓存或 DRAM 进行服务。这意味着这种内存访问的延迟会显著增加。内存子系统组件的典型延迟如表3所示。还有一个交互视图[50]，可视化了现代系统中不同操作的延迟。性能会受到严重影响，特别是当内存请求在最后一级缓存（LLC）中丢失并一直到达主存时。英特尔 ® Memory Latency Checker[51]（MLC）是用于测量内存延迟和带宽以及它们随系统负载增加而

[50] 交互延迟 - https://colin-scott.github.io/personal_website/research/interactive_latency.html

[51] Memory Latency Checker - https://www.intel.com/software/mlc

变化的工具。MLC 对于建立测试系统的基准和进行性能分析非常有用。当我们讨论 Section 4.10中的内存延迟和带宽时，我们将使用这个工具。

Table 3: x86 平台内存子系统的典型延迟。

内存层次结构组件	延迟（周期/时间）
L1 缓存	4 个周期（~1 纳秒）
L2 缓存	10-25 个周期（5-10 纳秒）
L3 缓存	~40 个周期（20 纳秒）
主内存	200 个周期以上（100 纳秒）

缓存失效可能会发生在指令和数据上。根据 Top-down Microarchitecture Analysis（见 Section 6.1），指令缓存（I-cache）失效被定义为前端停顿，而数据缓存（D-cache）失效被定义为后端停顿。指令缓存失效在 CPU 流水线的早期阶段（指令获取阶段）发生。数据缓存失效则发生在后期，即指令执行阶段。

Linux `perf`用户可以通过运行以下命令来收集 L1 缓存失效的数量：

```
$ perf stat -e mem_load_retired.fb_hit,mem_load_retired.l1_miss,
  mem_load_retired.l1_hit,mem_inst_retired.all_loads -- a.exe
   29580  mem_load_retired.fb_hit
   19036  mem_load_retired.l1_miss
  497204  mem_load_retired.l1_hit
  546230  mem_inst_retired.all_loads
```

以上是针对 L1 数据缓存和填充缓冲区的所有加载操作的细分。加载操作可能命中已分配的填充缓冲区（`fb_hit`），或者命中 L1 缓存（`l1_hit`），或者两者都未命中（`l1_miss`），因此 `all_loads = fb_hit + l1_hit + l1_miss`。我们可以看到，只有 3.5% 的所有加载操作在 L1 缓存中未命中，因此 L1 命中率为 96.5%。

我们可以进一步分析 L1 数据缺失并分析 L2 缓存行为，方法是运行：

```
$ perf stat -e mem_load_retired.l1_miss,
  mem_load_retired.l2_hit,mem_load_retired.l2_miss -- a.exe
  19521  mem_load_retired.l1_miss
  12360  mem_load_retired.l2_hit
   7188  mem_load_retired.l2_miss
```

从这个例子中，我们可以看到，在 L1 D-cache 中缺失的加载操作中有 37% 也在 L2 缓存中缺失，因此 L2 命中率为 63%。以类似的方式，可以对 L3 缓存进行细分。

4.8 错误预测的分支

现代 CPU 尝试预测分支指令的结果（是否被执行）。例如，当处理器看到这样的代码时：

```
dec eax
jz .zero
# eax 不为 0
...
zero:
# eax 为 0
```

在上面的例子中，`jz` 指令是一个分支。现代 CPU 架构试图预测每个分支的结果以提高性能。这被称为“推测执行”，我们在 Section 3.3.3中讨论过。处理器会假设，例如，分支不会被执行，并执行相应于 `eax` 不为 `0` 的情况的代码。然而，如果猜测错误，这被称为“分支预测错误”，CPU 需要撤销最近所做的所有推测工作。

错误预测的分支通常会导致 10 到 20 个时钟周期的惩罚。首先，根据错误预测获取和执行的所有指令都需要从流水线中清除。之后，一些缓冲区可能需要清理，以恢复从坏的推测开始的状态。最后，流水线需要等待确定正确的分支目标地址，这会导致额外的执行延迟。

Linux `perf` 用户可以通过运行以下命令来检查分支预测错误的数量：

```
$ perf stat -e branches,branch-misses -- a.exe
   358209  branches
    14026  branch-misses #    3.92% 的分支错误预测率
# 或者简单地执行：
$ perf stat -- a.exe
```

4.9 性能指标

除了本章前面讨论的性能事件外，性能工程师经常使用基于原始事件的指标。表4显示了针对英特尔第 12 代 Goldencove 架构的一系列指标，包括描述和公式。该列表并非详尽无遗，但显示了最重要的指标。有关英特尔 CPU 及其公式的完整指标列表可在TMA_metrics.xlsx中找到。Section 4.11展示了如何在实践中使用性能指标。

Table 4: 英特尔 Goldencove 架构的一系列次要指标及其描述和公式（非详尽）。

指标名称	描述	公式
L1MPKI	每千条已退役 (retired) 需求负载指令的 L1 缓存真未命中数量	1000*MEM_LOAD_RETIRED.L1_MISS_PS /INST_RETIRED.ANY
L2MPKI	每千条已退役 (retired) 需求负载指令的 L2 缓存真未命中数量	1000*MEM_LOAD_RETIRED.L2_MISS_PS /INST_RETIRED.ANY
L3MPKI	每千条已退役 (retired) 需求负载指令的 L3 缓存真未命中数量	1000*MEM_LOAD_RETIRED.L3_MISS_PS /INST_RETIRED.ANY
BranchMispr.Ratio	所有分支的误判率	BR_MISP_RETIRED.ALL_BRANCHES /BR_INST_RETIRED.ALL_BRANCHES
CodeSTLBMPKI S	TLB(2 级 TLB) 代码推测性未命中每千条指令	1000*ITLB_MISSES.WALK_COMPLETED /INST_RETIRED.ANY
LoadSTLBMPKI S	TLB 数据加载推测性 未命中每千条指令	1000*DTLB_LOAD_MISSES.WALK_COMPLETED /INST_RETIRED.ANY
StoreSTLBMPKI	STLB 数据存储推测性 未命中每千条指令	1000*DTLB_STORE_MISSES.WALK_COMPLETED /INST_RETIRED.ANY
LoadMissRealLatency	L1 数据缓存未命中 需求负载操作的实际平均延迟（核心周期）	L1D_PEND_MISS.PENDING /MEM_LOAD_COMPLETED.L1_MISS_ANY

指标名称	描述	公式
ILP	每内核指令级并行性（当存在执行时，平均执行的 μops 数量）	UOPS_EXECUTED.THREAD /UOPS_EXECUTED.CORE_CYCLES_GE_1 如果启用 SMT，则除以 2
MLP	每线程内存级并行性 (当至少有一个未命中时，L1 未命中需求负载的平均数量）	L1D_PEND_MISS.PENDING /L1D_PEND_MISS.PENDING_CYCLES
DRAMBWUse	平均外部内存带宽使用（读写 GB/秒）	(64*(UNC_M_CAS_COUNT.RD +UNC_M_CAS_COUNT.WR) /1GB)/时间
IpCall	近调用每条指令	INST_RETIRED.ANY /BR_INST_RETIRED.NEAR_CALL
IpBranch 每	分支指令	INST_RETIRED.ANY /BR_INST_RETIRED.ALL_BRANCHES
IpLoad	每加载指令	INST_RETIRED.ANY /MEM_INST_RETIRED.ALL_LOADS_PS
IpStore	每存储指令	INST_RETIRED.ANY /MEM_INST_RETIRED.ALL_STORES_PS
IpMisp	每非推测分支误判指令	INST_RETIRED.ANY /BR_MISP_RETIRED.ALL_BRANCHES
IpFLOP	每浮点 (FP) 操作指令	请参阅 TMA_metrics.xlsx
IpArithScalarSP	每标量单精度 FP 算术指令	INST_RETIRED.ANY /FP_ARITH_INST_RETIRED.SCALAR_SINGLE
IpArithScalarDP	每标量双精度 FP 算术指令	INST_RETIRED.ANY /FP_ARITH_INST_RETIRED.SCALAR_DOUBLE
IpArithAVX128	每 FP 算术 AVX128128 位指令	INST_RETIRED.ANY/(FP_ARITH_INST_RETIRED.128B_PACKED_DOUBLE + FP_ARITH_INST_RETIRED.128B_PACKED_SINGLE)
IpArithAVX256	每 FP 算术 AVX256256 位指令	INST_RETIRED.ANY/ (FP_ARITH_INST_RETIRED.256B_PACKED_DOUBLE + FP_ARITH_INST_RETIRED.256B_PACKED_SINGLE)
IpSWPF	每个软件预取指令 (of any type)	INST_RETIRED.ANY /SW_PREFETCH_ACCESS.T0:u0xF

关于这些指标的几点说明。首先，ILP（指令级并行性）和 MLP（内存级并行性）指标并不代表应用程序的理论最大值；相反，它们衡量的是在给定机器上应用程序的实际 ILP 和 MLP。在拥有无限资源的理想机器上，这些数字会更高。其次，除了“DRAM 带宽使用 (DRAM BW Use)”和“加载未命中实际延迟 (Load Miss Real Latency)”之外的所有指标都是分数；我们可以对它们进行相当直接的推理来判断特定指标是高还是低。但是，要理解“DRAM 带宽使用”和“加载未命中实际延迟”指标，我们需要将其放在上下文中考虑。对于前者，我们想知道程序是否饱和了内存带宽。后者给你一个缓存未命中的平均成本的概念，除非你知道缓存层次结构中每个组件的延迟，否则这个数字本身是没有用的。我们将在下一节讨论如何找出缓存延迟和峰值内存带宽。

一些工具可以自动报告性能指标。如果没有，你总可以手动计算这些指标，因为你知道公式和必须收集的相应性能事件。表4提供了针对英特尔 Goldencove 架构的公式，但只要底层的性能事件可用，你就可以在另一个平台上构建

类似的指标。

4.10 内存延迟和带宽

在现代环境中，低效的内存访问通常是主要的性能瓶颈。因此，处理器从内存子系统中获取单个字节的速度（延迟）以及每秒可以获取多少字节（带宽）是决定应用程序性能的关键因素之一。这两个方面在各种场景中都很重要，我们稍后将看到一些示例。在本节中，我们将专注于测量内存子系统组件的峰值性能。

在 x86 平台上，可以成为有用工具之一的是英特尔内存延迟检查器（MLC），[52]它在 Windows 和 Linux 上都可以免费使用。MLC 可以使用不同的访问模式和负载来测量缓存和内存的延迟和带宽。在基于 ARM 的系统上没有类似的工具，但是用户可以从源代码中下载并构建内存延迟和带宽基准测试。这类项目的示例包括lmbench[53]，bandwidth[54]和Stream。[55]

我们只关注一个子集指标，即空闲读取延迟和读取带宽。让我们从读取延迟开始。空闲表示在进行测量时，系统处于空闲状态。这将为我们提供从内存系统组件获取数据所需的最小时间，但是当系统被其他“内存消耗量大”的应用程序加载时，此延迟会增加，因为在各个点上可能会有更多的资源排队。MLC 通过进行相关加载（也称为指针追踪）来测量空闲延迟。一个测量线程分配一个非常大的缓冲区，并对其进行初始化，以便缓冲区内的每个（64 字节）缓存行包含指向该缓冲区内另一个非相邻缓存行的指针。通过适当调整缓冲区的大小，我们可以确保几乎所有的加载都命中某个级别的缓存或主存。

我们的测试系统是一台英特尔 Alderlake 主机，配备 Core i7-1260P CPU 和 16GB DDR4 @ 2400 MT/s 双通道内存。该处理器有 4 个 P（性能）超线程核心和 8 个 E（高效）核心。每个 P 核心有 48KB 的 L1 数据缓存和 1.25MB 的 L2 缓存。每个 E 核心有 32KB 的 L1 数据缓存，而四个 E 核心组成一个集群，可以访问共享的 2MB L2 缓存。系统中的所有核心都由 18MB 的 L3 缓存支持。如果我们使用一个 10MB 的缓冲区，我们可以确保对该缓冲区的重复访问会在 L2 中未命中，但在 L3 中命中。以下是示例 `mlc` 命令：

```
$ ./mlc --idle_latency -c0 -L -b10m
Intel(R) Memory Latency Checker - v3.10
Command line parameters: --idle_latency -c0 -L -b10m

Using buffer size of 10.000MiB
*** Unable to modify prefetchers (try executing 'modprobe msr')
*** So, enabling random access for latency measurements
Each iteration took 31.1 base frequency clocks (  12.5  ns)
```

选项 `--idle_latency` 测量读取延迟而不加载系统。MLC 具有 `--loaded_latency` 选项，用于在由其他线程生成的内存流量存在时测量延迟。选项 `-c0` 将测量线程固定在逻辑 CPU 0 上，该 CPU 位于 P 核心上。选项 `-L` 启用大页以限制我们的测量中的 TLB 效应。选项 `-b10m` 告诉 MLC 使用 10MB 缓冲区，在我们的系统上可以放在 L3 缓存中。

图 26 显示了 L1、L2 和 L3 缓存的读取延迟。图中有四个不同的区域。从 1KB 到 48KB 缓冲区大小的左侧的第一个区域对应于 L1d 缓存，该缓存是每个物理核心私有的。我们可以观察到 E 核心的延迟为 0.9ns，而 P 核心稍高为 1.1ns。此外，我们可以使用此图来确认缓存大小。请注意，当缓冲区大小超过 32KB 时，E 核心的延迟开始上升，但是在 48KB 之前 P 核心的延迟保持不变。这证实了 E 核心的 L1d 缓存大小为 32KB，而 P 核心的 L1d 缓存大小为 48KB。

第二个区域显示 L2 缓存延迟，E 核的延迟几乎是 P 核的两倍（5.9ns vs. 3.2ns）。对于 P 核，延迟在我们超过 1.25MB 缓冲区大小后会增加，这是预期的。但我们期望 E 核的延迟保持不变，直到 2MB，但在我们的测量中没有发生这种

[52] Intel MLC 工具 - https://www.intel.com/content/www/us/en/download/736633/intel-memory-latency-checker-intel-mlc.html

[53] lmbench - https://sourceforge.net/projects/lmbench

[54] Zack Smith 的内存带宽基准测试 - https://zsmith.co/bandwidth.php

[55] Stream - https://github.com/jeffhammond/STREAM

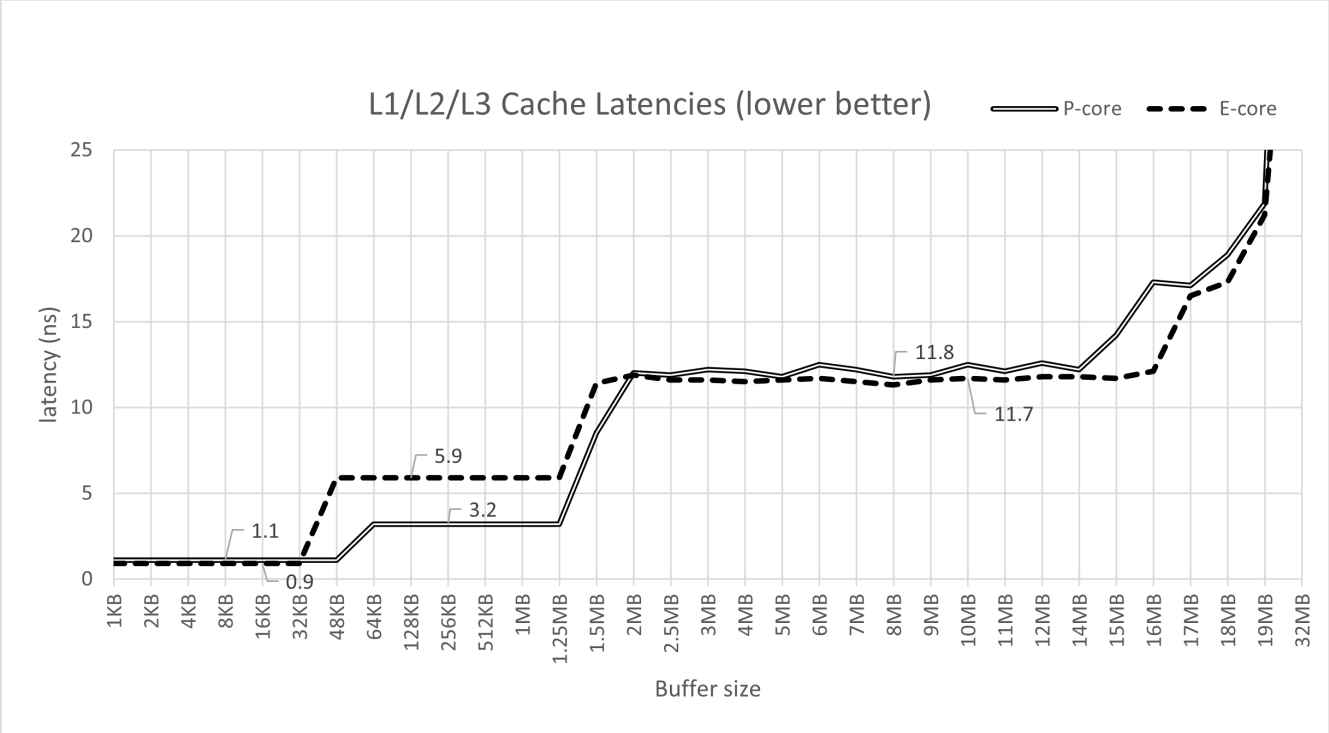


Figure 26: 在 Intel Core i7-1260P 上使用 mlc 工具测量的 L1/L2/L3 缓存读取延迟，启用了大页。

情况。

第三个区域从 2MB 到 14MB 对应于 L3 缓存延迟，对于两种类型的内核都大约为 12ns。系统中所有内核共享的 L3 缓存的总大小为 18MB。有趣的是，我们从 15MB 开始看到一些意想不到的动态，而不是 18MB。这很可能是因为一些访问错过了 L3，需要去主内存。

第四个区域对应于内存延迟，图表上只显示了其开始部分。当我们越过 18MB 的边界时，延迟会急剧上升，并在 E 核心的 24MB 和 P 核心的 64MB 处开始趋于稳定。使用更大的缓冲区大小为 500MB 时，E 核心的访问延迟为 45ns，P 核心为 90ns。这测量了内存延迟，因为几乎没有加载会命中 L3 缓存。

使用类似的技术，我们可以测量内存层次结构的各个组件的带宽。为了测量带宽，MLC 执行的加载请求不会被任何后续指令使用。这允许 MLC 生成可能的最大带宽。MLC 在每个配置的逻辑处理器上生成一个软件线程。每个线程访问的地址是独立的，线程之间没有数据共享。与延迟实验一样，线程使用的缓冲区大小确定了 MLC 是在测量 L1/L2/L3 缓存带宽还是内存带宽。

```
./mlc --max_bandwidth -k0-15 -Y -L -b10m
Measuring Maximum Memory Bandwidths for the system
Bandwidths are in MB/sec (1 MB/sec = 1,000,000 Bytes/sec)
Using all the threads from each core if Hyper-threading is enabled
Using traffic with the following read-write ratios
ALL Reads        :      33691.53
3:1 Reads-Writes :      30352.45
2:1 Reads-Writes :      29722.28
1:1 Reads-Writes :      28382.45
Stream-triad like:      30503.68
```

这里的新选项是 `-k`，它指定了用于测量的 CPU 编号列表。`-Y` 选项告诉 MLC 使用 AVX2 加载，即每次加载 32 字节。

MLC 使用不同的读写比例来测量带宽，但在下图中，我们只显示了全部读取带宽，因为它可以让我们对内存带宽的峰值有一个直观的了解。但其他比例也可能很重要。我们在使用 Intel MLC 测量的系统的组合延迟和带宽数字如图 27 所示。

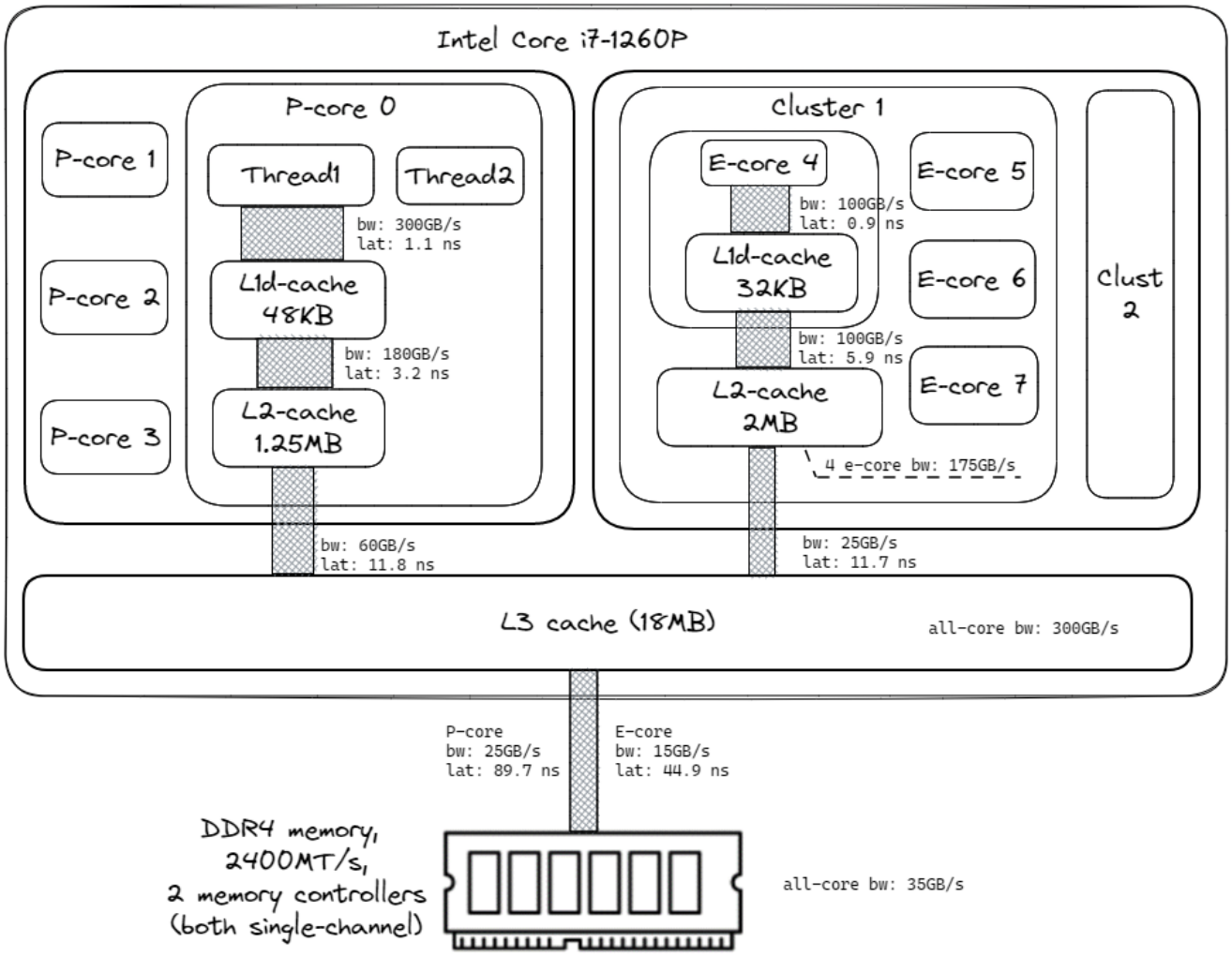


Figure 27: Intel Core i7-1260P 内存层次结构的块图和外部 DDR4 内存。

核心可以从较低级别的缓存（如 L1 和 L2）中获得比从共享的 L3 缓存或主内存中更高的带宽。共享缓存（如 L3 和 E 核心 L2）相当好地扩展，可以同时为多个核心提供请求。例如，单个 E 核心 L2 的带宽为 100GB/s。使用来自同一集群的两个 E 核心，我测量了 140GB/s，三个 E 核心为 165GB/s，而所有四个 E 核心可以从共享 L2 中获得 175GB/s。对于 L3 缓存也是如此，单个 P 核心的带宽为 60GB/s，而单个 E 核心只有 25GB/s。但是当所有核心都被使用时，L3 缓存可以维持 300GB/s 的带宽。

请注意，我们用纳秒测量延迟，用 GB/s 测量带宽，因此它们还取决于核心运行的频率。在各种情况下，观察到的数字可能不同。例如，假设当仅在系统上以最大睿频运行时，P 核心的 L1 延迟为 `X`，L1 带宽为 `Y`。当系统负载满时，我们可能观察到这些指标分别变为 `1.25X` 和 `0.75Y`。为了减轻频率效应，与其使用纳秒，延迟和度量可以使用核心周期来表示，并归一化为一些样本频率，比如 3Ghz。

了解计算机的主要特征是评估程序如何利用可用资源的基本方法。当我们讨论 Roofline 性能模型时，我们将在 Section 5.6返回到这个主题。如果您经常在单个平台上分析性能，最好记住内存层次结构的各个组件的延迟和带宽，或者将它们随时备查。这有助于建立对测试系统的心理模型，将有助于您进一步的性能分析，正如您将在接下来看到的那样。

4.11 案例研究：分析四个基准测试的性能指标

在本章中讨论的所有内容综合起来，我们运行了来自不同领域的四个基准测试，并计算了它们的性能指标。首先，让我们介绍这些基准测试。

1. Blender 3.4 - 一个开源的 3D 创建和建模软件项目。这个测试是使用 Blender 的 Cycles 性能进行的，使用了 BMW27 混合文件。使用了所有的硬件线程。URL: https://download.blender.org/release。命令行：`./blender -b bmw27_cpu.blend -noaudio --enable-autoexec -o output.test -x 1 -F JPEG -f 1`。
2. Stockfish 15 - 一个先进的开源国际象棋引擎。这个测试是一个内置的 stockfish 基准测试。只使用了一个硬件线程。URL: https://stockfishchess.org。命令行：`./stockfish bench 128 1 24 default depth`。
3. Clang 15 自我构建 - 这个测试使用 clang 15 从源代码构建 clang 15 编译器。使用了所有的硬件线程。URL: https://www.llvm.org。命令行：`ninja -j16 clang`。
4. CloverLeaf 2018 - 一个拉格朗日-欧拉流体动力学基准测试。使用了所有的硬件线程。这个测试使用了 clover_bm.in 输入文件（问题 5）。URL: http://uk-mac.github.io/CloverLeaf。命令行：`./clover_leaf`。

为了进行这个练习，我们在具有以下特征的机器上运行了所有四个基准测试：

- 12 代 Alderlake Intel(R) Core(TM) i7-1260P CPU @ 2.10GHz (4.70GHz Turbo)，4P+8E 核心，18MB L3 缓存
- 16 GB DDR4 @ 2400 MT/s 内存
- 256GB NVMe PCIe M.2 SSD
- 64 位 Ubuntu 22.04.1 LTS (Jammy Jellyfish)

为了收集性能指标，我们使用了`toplev.py`脚本，它是pmu-tools[56]的一部分，由 Andi Kleen 编写：

```
$ ~/workspace/pmu-tools/toplev.py -m --global --no-desc -v -- <app with args>
```

表5提供了我们四个基准测试的性能指标的并排比较。通过查看这些指标，我们可以了解这些工作负载的性质。在收集性能优化分析文件并深入研究这些应用程序的代码之前，我们可以对这些基准测试做出一些假设。

- **Blender**。工作在 P 核心和 E 核心之间相当均匀分配，两种核心类型的 IPC 都相当不错。每千条指令的缓存未命中次数相当低（见`L*MPKI`）。分支误预测对性能有所影响：分支误预测比例指标为`2%`；我们每`610`条指令就有一个误预测（见`IpMispredict`指标），这个数值并不糟糕，但也不完美。TLB 并不是瓶颈，因为我们在 STLB 中很少发生未命中。我们忽略加载未命中延迟指标，因为缓存未命中的数量非常低。ILP 相当高。Goldencove 是一个 6 宽度的体系结构；ILP 为`3.67`意味着算法几乎每个周期利用了核心资源的`2/3`。内存带宽需求很低，只有 1.58 GB/s，远低于该机器的理论最大值。从`Ip*`指标来看，我们可以得知 Blender 是一个浮点算法（见`IpFLOP`指标），其中有很大一部分是向量化的浮点运算（见`IpArith AVX128`）。但是，算法的某些部分也是非向量化的标量浮点单精度指令（`IpArith Scal SP`）。另外，请注意每 90 条指令就会有一个明确的软件内存预取（`IpSWPF`）；我们期望在 Blender 的源代码中看到这些提示。结论：Blender 的性能受到 FP 计算的限制，偶尔会出现分支误预测。

- **Stockfish**。我们只使用了一个硬件线程运行它，因此 E 核心上没有任何工作，这是预期的。L1 缓存未命中的数量相对较高，但大部分都包含在 L2 和 L3 缓存中。分支误预测比例很高；我们每`215`条指令就会付出一次误预测的代价。我们可以估计，我们每`215`（指令）`/ 1.80`（IPC）`= 120`个周期就会发生一次误预测，这是非常频繁的。与 Blender 的推理类似，我们可以说 TLB 和 DRAM 带宽对 Stockfish 不构成问题。进一步分析，我们发现工作负载中几乎没有 FP 操作。结论：Stockfish 是一个整数计算工作负载，受分支误预测的影响很大。

- **Clang 15 自我构建**。C++ 代码编译是一项性能特性非常平坦的任务，即没有大的热点。通常，您会发现运行时间归因于许多不同的函数。我们首先注意到的是，P 核心比 E 核心多做了 68% 的工作，并且 IPC 要好 42%。但是 P 核心和 E 核心的 IPC 都很低。乍一看，`L*MPKI`指标看起来并不令人担忧；然而，结合加载未命中实际

[56] pmu-tools - https://github.com/andikleen/pmu-tools

延迟（LdMissLat，以核心时钟表示），我们可以看到缓存未命中的平均成本相当高（~77 个周期）。现在，当我们查看*STLB MPKI指标时，我们注意到与我们测试的任何其他基准测试都存在实质性差异。这是由于 Clang 编译器（以及其他编译器）的另一个方面：二进制文件的大小相对较大（超过 100 MB）。代码不断跳转到远处的位置，导致 TLB 子系统的压力很大。正如您所看到的，该问题存在于指令（请参阅代码STLB MPKI）和数据（请参阅加载STLB MPKI）之间。让我们继续进行分析。DRAM 带宽使用率高于前两个基准测试，但仍然没有达到我们平台的最大内存带宽的一半（约为 25 GB/s）。我们关注的另一个问题是每次调用的指令数量非常少（IpCall）：每个函数调用只有约 41 条指令。不幸的是，这是编译代码库的本质：它有数千个小函数。编译器需要更积极地内联所有这些函数和包装器。然而，我们怀疑与进行函数调用相关的性能开销仍然是 Clang 编译器的一个问题。此外，人们可以注意到高ipBranch和IpMispredict指标。对于 Clang 编译，每五条指令中就有一条分支，大约每 35 条分支中就有一条误预测。几乎没有 FP 或向量指令，但这并不奇怪。结论：Clang 具有庞大的代码库，平坦的性能优化分析文件，许多小函数和“分支”代码；性能受到数据缓存和 TLB 未命中以及分支误预测的影响。

- **CloverLeaf**。与之前一样，我们从分析指令和核心周期开始。P 核心和 E 核心完成的工作量大致相同，但 P 核心需要更长的时间来完成这项工作，导致 P 核心上的一个逻辑线程的 IPC 比一个物理 E 核心上的 IPC 低。我们对此还没有一个很好的解释。L*MPKI指标很高，特别是每千条指令的 L3 未命中次数。加载未命中延迟（LdMissLat）超出了图表范围，表明平均缓存未命中的价格非常高。接下来，我们看一下DRAM带宽使用指标，发现内存带宽完全饱和了。这就是问题所在：系统中的所有核心共享同一个内存总线，因此它们竞争访问主存，有效地阻塞了执行。CPU 缺乏它们需要的数据。进一步说，我们可以看到 CloverLeaf 几乎没有受到分支误预测或函数调用开销的影响。指令混合主要由 FP 双精度标量操作主导，代码的某些部分被向量化。结论：多线程 CloverLeaf 受到内存带宽的限制。

Table 5: 四个基准测试的性能指标。

指标名称	核心类型	Blender	Stockfis	h Clang15 -selfbuild	CloverLeaf
指令数	P-核心	6.02E+12	6.59E+11	2.40E+13	1.06E+12
核心周期数	P-核心	4.31E+12	3.65E+11	3.78E+13	5.25E+12
IPC	P-核心	1.40	1.80	0.64	0.20
CPI	P-核心	0.72	0.55	1.57	4.96
指令数	E-核心	4.97E+12	0	1.43E+13	1.11E+12
核心周期数	E-核心	3.73E+12	0	3.19E+13	4.28E+12
IPC	E-核心	1.33	0	0.45	0.26
CPI	E-核心	0.75	0	2.23	3.85
L1MPKI	P-核心	3.88	21.38	6.01	13.44
L2MPKI	P-核心	0.15	1.67	1.09	3.58
L3MPKI	P-核心	0.04	0.14	0.56	3.43
分支错误率	E-核心	0.02	0.08	0.03	0.01
代码 STLB MPKI	P-核心	0	0.01	0.35	0.01
加载 STLB MPKI	P-核心	0.08	0.04	0.51	0.03
存储 STLB MPKI	P-核心	0	0.01	0.06	0.1
加载缺失延迟（时钟）	P-核心	12.92	10.37	76.7	253.89
ILP	P-核心	3.67	3.65	2.93	2.53
MLP	P-核心	1.61	2.62	1.57	2.78

指标名称	核心类型	Blender	Stockfis h Clang15 -selfbuild		CloverLeaf
DRAM 带宽 (GB/s)	全部	1.58	1.42	10.67	24.57
IpCall	全部	176.8	153.5	40.9	2,729
IpBranch	全部	9.8	10.1	5.1	18.8
IpLoad	全部	3.2	3.3	3.6	2.7
IpStore	全部	7.2	7.7	5.9	22.0
IpMispredict	全部	610.4	214.7	177.7	2,416
IpFLOP	全部	1.1	1.82E+06	286,348	1.8
IpArith	全部	4.5	7.96E+06	268,637	2.1
IpArith Scal SP	全部	22.9	4.07E+09	280,583	2.60E+09
IpArith Scal DP	全部	438.2	1.22E+07	4.65E+06	2.2
IpArith AVX128	全部	6.9	0.0	1.09E+10	1.62E+09
IpArith AVX256	全部	30.3	0.0	0.0	39.6
IpSWPF	全部	90.2	2,565	105,933	172,348

正如您从这项研究中看到的，仅仅通过查看指标就可以了解很多关于程序行为的信息。它回答了“是什么？”的问题，但没有告诉你“为什么？”。为此，您需要收集性能优化分析文件，我们将在以后的章节中介绍。本书的第二部分将讨论如何减轻我们分析的四个基准测试中可能出现的性能问题。

请记住，表 5 中的性能指标摘要只告诉您程序的平均行为。例如，我们可能看到 CloverLeaf 的 IPC 为 0.2，而实际上它可能永远不会以这样的 IPC 运行，相反它可能有两个持续时间相同的阶段，一个以 IPC 0.1 运行，另一个以 IPC 0.3 运行。性能工具通过为每个指标报告统计数据和平均值来解决这个问题。通常，最小值、最大值、第 95 个百分位数和方差 (stdev/avg) 足以了解分布。此外，一些工具允许绘制数据，因此您可以看到特定指标的值在程序运行期间如何变化。例如，图 28 显示了 CloverLeaf 基准测试中 IPC、L*MPKI、DRAM BW 和平均频率的动态变化。“pmu-tools”软件包可以在您添加 `--xlsx` 和 `--xchart` 选项后自动生成这些图表。

```
$ ~/workspace/pmu-tools/toplev.py -m --global --no-desc -v --xlsx workload.xlsx  –xchart --
    ./clover_leaf
```

尽管与摘要中报告的值偏差不大，但我们可以看到工作负载并不总是稳定的。在查看 IPC 图表后，我们可以假设工作负载中没有不同的阶段，变化是由性能事件的多路复用引起的（在 Section 5.3 中讨论）。然而，这只是一个需要证实或否定的假设。可能的方法是通过以更高粒度（在本例中为 10 秒）运行收集来收集更多数据点并研究源代码。仅根据数字得出结论要小心；始终获取第二个数据源来确认您的假设。

总之，查看性能指标有助于构建关于程序中发生了什么和没有发生什么的正确思维模型。深入分析，这些数据将对您大有裨益。

问题和练习

1. CPU 核心时钟与参考时钟之间的区别是什么？
2. 退役指令与执行指令之间的区别是什么？
3. 当你提高频率时，IPC（每个周期内的指令数）是上升、下降还是保持不变？
4. 查看表4中的`DRAM BW Use`公式。你认为为什么会有一个常数64？
5. 使用 MLC、stream 或其他工具，测量你用于开发/基准测试的机器上的缓存层次结构和内存的带宽和延迟。
6. 运行你日常使用的应用程序。收集性能指标。有什么让你感到惊讶的吗？

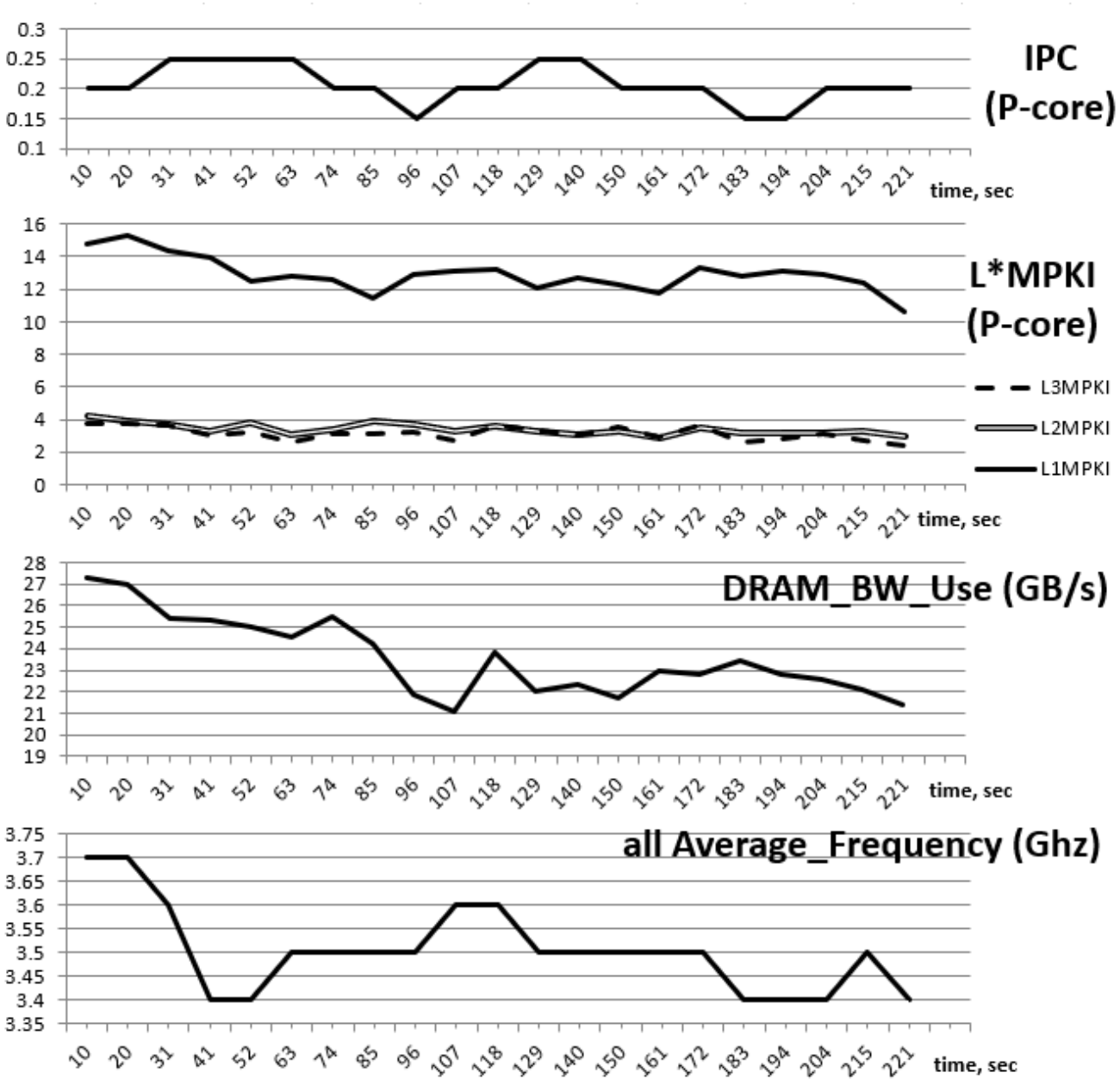


Figure 28: CloverLeaf 基准测试的一组指标图表

AI 回答：

1. **CPU 核心时钟与参考时钟的区别：**
   - CPU 核心时钟是 CPU 实际运行的频率，它决定了 CPU 执行指令的速度。这个频率可能会因为 CPU 的动态频率调整技术（如 Intel 的 Turbo Boost）而变化，以适应不同的工作负载。
   - 参考时钟是一个固定的时钟信号，通常由外部时钟发生器提供，它为 CPU 提供了一个基准频率。在没有动态频率调整的情况下，核心时钟和参考时钟的频率是相同的。
2. **退役指令与执行指令的区别：**
   - 退役指令（Retired Instructions）是指 CPU 已经完成执行并提交结果的指令。这些指令已经通过了所有的执行阶段，包括执行、访存和写回结果。
   - 执行指令（Executed Instructions）是指 CPU 正在处理的指令，它们可能还在执行阶段，尚未完成。在现代处理器中，由于推测执行的存在，执行的指令数量可能会超过退役的指令数量。
3. **提高频率时 IPC 的变化：**
   - IPC（Instructions Per Clock，每个时钟周期内的指令数）是一个衡量 CPU 效率的指标，它与 CPU 的微架构设计有关。理论上，当你提高 CPU 的频率时，IPC 应该保持不变，因为 IPC 不受时钟频率的影响。然而，在实际情况中，提高频率可能会影响 CPU 的缓存命中率、分支预测的准确性等，从而间接影响 IPC。
4. **`DRAM BW Use`公式中的常数64：**
   - 在`DRAM BW Use`（动态随机存取内存带宽使用）公式中，常数64通常与内存数据总线的宽度有关。在许多现代计算机系统中，内存总线宽度为 64 位（或 8 字节），这意味着每个内存时钟周期可以传输 64 位的数据。这个常数用于将内存操作的原始计数转换为更易于理解的单位，如 GB/s。
5. **测量缓存层次结构和内存的带宽和延迟：**
   - 使用 MLC（Memory Latency Checker）、stream 或其他性能分析工具可以测量机器上的缓存层次结构和内存的带宽和延迟。这些工具通过生成特定的内存访问模式来评估系统在不同负载下的性能。例如，MLC 可以测量在空闲状态下的内存延迟，以及在不同读写比例下的内存带宽。
6. **日常应用程序的性能指标收集：**
   - 运行日常使用的应用程序并收集性能指标可以帮助了解应用程序在实际环境中的表现。可能会发现一些出乎意料的行为，例如，某些应用程序在特定条件下可能会有较高的 CPU 利用率或内存带宽需求，这可能揭示了性能瓶颈或优化机会。这些发现可以指导开发者进行性能优化，或者在系统设计时做出更明智的决策。

容量规划练习：假设你是案例研究中四个应用程序的所有者。公司管理层要求你为每个应用程序建立一个小型计算农场，主要目标是最大化性能（吞吐量）。你得到的预算有限，但足以购买 1 台中档服务器系统（Mac Studio、Supermicro/Dell/HPE 服务器机架等）或 1 台高端台式机（带超频 CPU、液冷、顶级 GPU、快速 DRAM），总共 4 台机器。这些可以是完全不同的四种系统。此外，你可以用这些钱购买 3-4 台低端系统，选择权在你。管理层希望每个应用程序的预算控制在 10,000 美元以内，但如果你能证明这笔开支是合理的，他们愿意灵活调整（10-20%）。假设 Stockfish 仍然是单线程的。再次查看这四个应用程序的性能特征，并为每个工作负载写下你会购买的计算机部件（CPU、内存、如有必要的独立 GPU）。你会优先考虑哪些规格参数？你会在哪些部分选择最昂贵的部件，又在哪里可以节省资金？尽可能详细地描述，搜索网络以找到确切的组件及其价格。考虑系统的所有组成部分：主板、硬盘驱动器、冷却解决方案、电源供应单元、机架/机箱/塔式等。你会运行哪些额外的性能实验来指导你的决策？

chatGPT 回答建议：

在进行容量规划时，首先需要了解每个应用程序的性能特征和需求。然后，需要考虑可用预算以及性能优化的可能性。下面是对每个应用程序的考虑以及可能的硬件选择：

1. **Stockfish（国际象棋引擎）：**
   - **性能特征**：CPU 密集型，单线程。
   - **建议硬件**：

- CPU：选择单线程性能最高的处理器，如 Intel Core i9-12900K 或 AMD Ryzen 9 5950X。价格大约在 600-800 美元之间。
- 内存：8GB DDR4 3200MHz 单通道内存足够，价格约为 50-70 美元。
- GPU：对 Stockfish 这样的单线程应用来说，独立 GPU 并不必要。
- 主板：选择兼容选定 CPU 的主板，价格约为 150-200 美元。
- 存储：250GB SATA SSD 足以满足需求，价格约为 40-60 美元。
- 冷却解决方案：使用空气冷却器即可，价格约为 30-50 美元。
- 电源供应单元：选择 550W 80+ Bronze 认证的电源，价格约为 50-70 美元。
- 机箱：选择价格适中的 ATX 机箱，价格约为 50-80 美元。

2. **Photoshop（图像处理软件）**：
   - **性能特征**：CPU 和内存密集型，对 GPU 加速有一定需求。
   - **建议硬件**：
     - CPU：选择多核性能较高的处理器，如 Intel Core i9-12900K 或 AMD Ryzen 9 5900X。价格约为 600-800 美元。
     - 内存：16GB DDR4 3600MHz 双通道内存，价格约为 120-150 美元。
     - GPU：选择带有至少 8GB 显存的显卡，如 NVIDIA RTX 3060 Ti 或 AMD Radeon RX 6700 XT。价格约为 400-500 美元。
     - 主板：选择兼容选定 CPU 和 GPU 的主板，价格约为 200-250 美元。
     - 存储：500GB NVMe SSD，价格约为 70-100 美元。
     - 冷却解决方案：对 CPU 使用液冷解决方案，价格约为 100-150 美元。
     - 电源供应单元：选择 650W 80+ Gold 认证的电源，价格约为 80-100 美元。
     - 机箱：选择具有良好散热设计和良好外观的中档机箱，价格约为 100-150 美元。
3. **TensorFlow（深度学习框架）**：
   - **性能特征**：GPU 加速的深度学习任务，对大量显存和高性能计算能力有需求。
   - **建议硬件**：
     - CPU：选择具有良好多核性能的处理器，如 Intel Core i9-12900K 或 AMD Ryzen 9 5900X。价格约为 600-800 美元。
     - 内存：32GB DDR4 3600MHz 双通道内存，价格约为 200-250 美元。
     - GPU：选择至少具有 12GB 显存的显卡，如 NVIDIA RTX 3080 或 AMD Radeon RX 6900 XT。价格约为 800-1000 美元。
     - 主板：选择兼容选定 CPU 和 GPU 的主板，价格约为 250-300 美元。
     - 存储：1TB NVMe SSD，价格约为 100-150 美元。
     - 冷却解决方案：对 CPU 使用高性能液冷解决方案，价格约为 150-200 美元。
     - 电源供应单元：选择 750W 80+ Platinum 认证的电源，价格约为 150-200 美元。
     - 机箱：选择具有良好散热设计和足够空间的高端机箱，价格约为 200-250 美元。
4. **WordPress（网站搭建工具）**：
   - **性能特征**：CPU 和存储密集型，对 GPU 无要求。
   - **建议硬件**：
     - CPU：选择具有良好多核性能的处理器，如 Intel Core i7-12700K 或 AMD Ryzen 7 5800X。价格约为 400-500 美元。
     - 内存：16GB DDR4 3200MHz 双通道内存，价格约为 120-150 美元。
     - GPU：无需独立显卡。
     - 主板：选择兼容选定 CPU 的主板，价格约为 150-200 美元。
     - 存储：500GB NVMe SSD，价格约为 70-100 美元。
     - 冷却解决方案：使用空气冷却器即可，价格约为 30-50 美元。

  - 电源供应单元：选择 550W 80+ Bronze 认证的电源，价格约为 50-70 美元。
  - 机箱：选择价格适中的 ATX 机箱，价格约为 50-80 美元。

对于硬件选择，我会优先考虑满足应用程序性能需求的部件，并在需要时选择性能更高的部件。我会在 CPU 和内存方面投入较多资金，因为它们对于大多数应用程序来说是关键的。另外，针对需要 GPU 加速的应用程序，我会选择具有良好性能和大量显存的显卡。在其他方面，我会尽量选择性价比高的部件，以节省预算。

为了指导我的决策，我可能会运行一些额外的性能实验，例如基准测试和负载测试，以评估不同硬件组合的性能表现。我还可能参考专业评测和用户评价，了解各个硬件组件的性能和可靠性。最终的决策将基于综合考虑性能、价格和可用预算。

章节总结

- 在本章中，我们介绍了性能分析中的基本指标，例如已执行/已完成指令、CPU 利用率、IPC/CPI、微操作、流水线槽、核心/参考时钟、缓存未命中和分支预测错误。我们展示了如何使用 Linux perf 采集这些指标中的每一个。
- 对于更高级的性能分析，可以收集许多派生指标。例如，MPKI（每千条指令的未命中）、Ip*（每个函数调用、分支、加载等指令）、ILP、MLP 等。本章的案例研究展示了如何通过分析这些指标获得可操作的见解。但是，仅查看汇总数字就得出结论要谨慎。不要陷入“电子表格性能工程”的陷阱，即只收集性能指标而不查看代码。总是寻求第二个数据源（例如，稍后讨论的性能优化分析文件）来验证您的假设。
- 内存带宽和延迟是当今许多生产软件包性能的关键因素，包括人工智能、高性能计算、数据库和许多通用应用程序。内存带宽取决于 DRAM 速度 (MT/s) 和内存通道数。现代高端服务器平台拥有 8-12 个内存通道，整个系统的带宽可达 500 GB/s，单线程模式下可达 50 GB/s。如今的内存延迟变化不大，事实上，随着新一代 DDR4 和 DDR5 的出现，它还在略微恶化。大多数现代系统的内存访问延迟都在 70-110 纳秒范围内。